StreamCast

A Complete Beginner's Guide
Backend Microservices + Angular Frontend

Prepared for: Afrin Banua
Project: StreamCast — Final Microservice
Date: 19 May 2026

Table of Contents

6. The User Journey End-to-End (Login → Title → Schedule)
7. How to Run It Locally
8. Common Troubleshooting
9. What to Learn Next — A Roadmap

1. What is StreamCast?

StreamCast is a **digital content distribution platform**. Think of it as the back-office software a streaming company (Netflix, Hotstar, Disney+) would use internally to:

- Catalog every movie, episode, or audio track they own (*Titles*).
- Track legal licensing agreements with content owners (*Contracts* and the *Clauses* inside them).
- Manage the external companies they ship content to (*Partners* — e.g. an Indian broadcaster, a regional CDN).
- Build delivery packages called *Manifests* and ship them to those partners (*Distribution-Delivery*).
- Plan when each title is broadcast/streamed (*Schedule*) and warn when two schedules clash (*Conflict*).
- Report on views and revenue (*Usage*).

The project is built as a **microservice system** — instead of one giant program, the responsibilities above are split into **10 separate Java/Spring Boot applications**, plus **1 Angular web frontend**. Each piece runs on its own port, has its own database, and talks to the others over the network.

> **One sentence summary:** StreamCast is a Spring-Boot microservices backend + Angular 17 frontend for managing content, licensing, scheduling, delivery, and analytics in a streaming/broadcasting business.

2. Core Concepts You Must Know First

Before touching the code, make these mental models stick. Everything in StreamCast is built on top of them.

2.1 What is a Microservice?

A **monolith** is one big application where every feature lives in one codebase and runs as one process. A **microservice architecture** splits that one big thing into many small applications. Each:

- Owns one business capability (e.g. just contracts, just scheduling).
- Has its **own database** — no one else reads its tables directly.
- Runs as its **own process**, often on its own port.
- Talks to other services only via **HTTP REST** (or messaging).

Why? Independent deploys, fault isolation, technology freedom per service. **Cost?** Networking complexity, distributed-system pain (service discovery, retries, transactions).

2.2 The Client–Server Model

The **client** is the program the user sees (here: the Angular app in the browser). The **server** is the program that holds data and rules (here: the Spring Boot services). The client sends a **request**, the server sends back a **response**. They speak over **HTTP**.

2.3 REST API basics

REST = an architectural style for building HTTP APIs. The core idea: every "thing" is a **resource** with a URL, and you act on it with a **verb**:

HTTP method	What it means	Example used in StreamCast
`GET`	Read data	`GET /api/titles` → list all titles
`POST`	Create new	`POST /contracts` → create a new contract
`PUT`	Replace existing	`PUT /api/titles/{id}` → update a title
`PATCH`	Partial update	`PATCH /api/schedules/{id}`
`DELETE`	Remove	`DELETE /api/titles/{id}`

2.4 JSON, HTTP status codes

Data is exchanged as **JSON** — a text format for objects and arrays.

```
{
  "name": "Inception",
  "genre": "Sci-Fi",
  "releaseDate": "2010-07-16"
}
```

Every HTTP response carries a status code:

- **2xx — Success.** 200 OK, 201 Created.
- **3xx — Redirect.**
- **4xx — You (client) did something wrong.** 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found.
- **5xx — Server broke.** 500 Internal Server Error, 503 Service Unavailable.

2.5 JWT — How StreamCast knows who you are

After you log in, the backend creates a **JWT** (a long signed string) that proves your identity. The browser stores it (in `localStorage`) and attaches it to *every* later request as:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs....
```

A JWT has three parts separated by dots: `header.payload.signature`. The middle *payload* is base64-encoded JSON like:

```
{
  "sub": "afrin@example.com",
  "role": "ADMIN",
  "exp": 1730000000
}
```

The **signature** is computed with a secret key only the server knows, so nobody can forge or tamper with a token. In StreamCast the API-Gateway validates the JWT on every request before forwarding it to a downstream service.

2.6 Databases & JPA / Hibernate

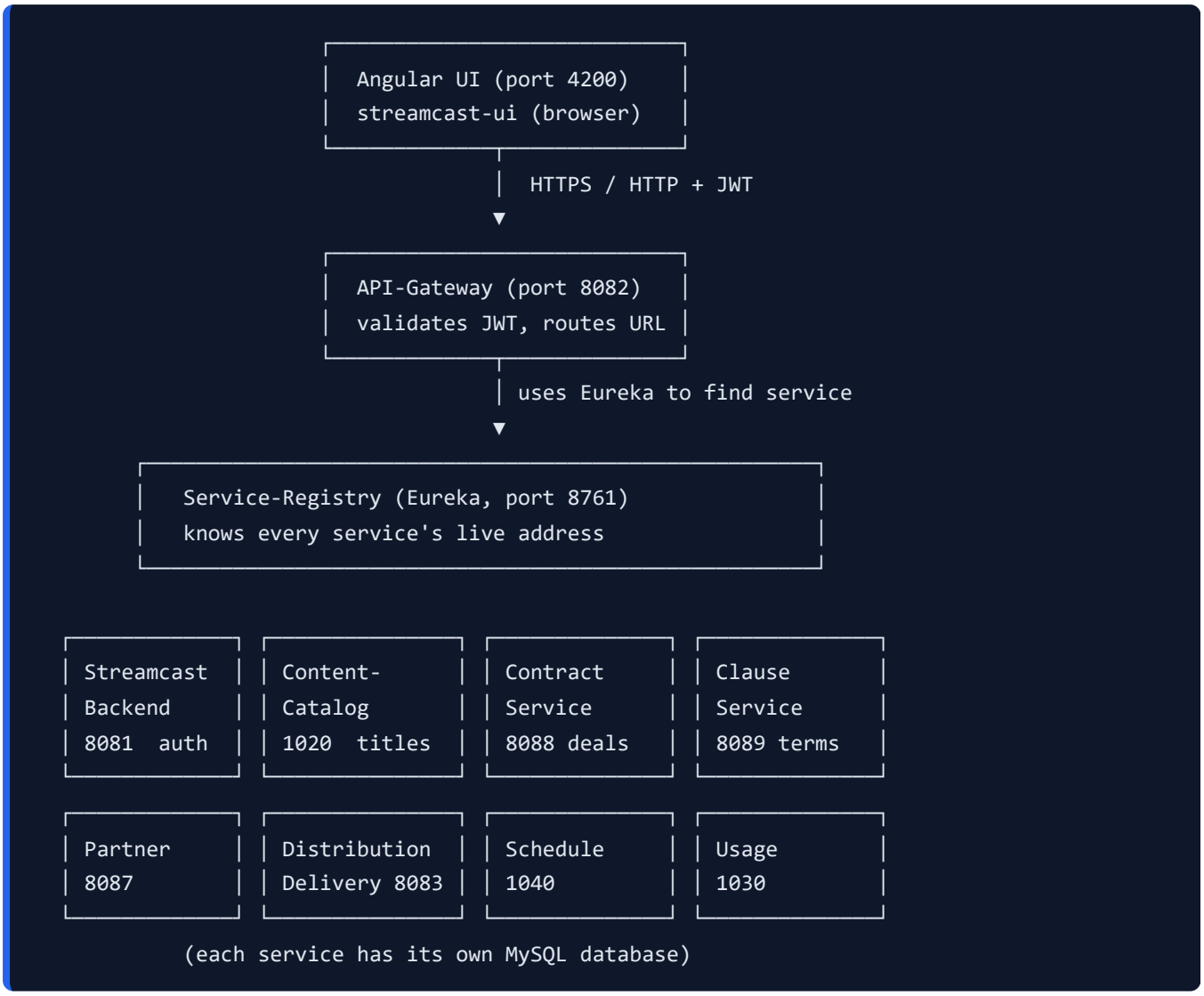
Every backend service stores its data in a **MySQL** database. Java code does not write raw SQL — it uses **JPA (Java Persistence API)**, implemented by Hibernate. You map a Java class to a table with annotations:

```
@Entity
@Table(name = "titles")
public class Title {
    @Id @GeneratedValue
    private Integer id;
    private String name;
    private LocalDate releaseDate;
    // ...
}
```

Then a `JpaRepository<Title, Integer>` gives you `findAll()`, `findById()`, `save()`, `deleteById()` for free — no SQL written.

3. The StreamCast Architecture Map

Here is the bird's-eye view of every piece and how requests flow:



Service summary table

Service	Port	Java	What it owns
Service-Registry	8761	17	Eureka — discovery
API-Gateway	8082	17	Single entry point, JWT validation
Streamcast-Backend	8081	17	Auth, users, roles, email
Content-Catalog-Service	1020	17	Titles, assets, metadata
Contract-Service	8088	21	Licensing contracts
Clause-Service	8089	21	Clauses inside contracts
Partner-Service	8087	21	Distribution partners
Distribution-Delivery	8083	17	Manifests, delivery attempts
schedule-service	1040	17	Schedules + conflicts
usage-service	1030	17	Views & revenue reports
streamcast-ui	4200	—	Angular 17 frontend

4. Backend Deep Dive

4.1 Java & Spring Boot in 5 minutes

Java is the language. Classes hold data + methods, types are checked at compile time, and you compile `.java` source into `.class` bytecode that runs on the JVM.

Spring is a framework that does the boring plumbing for you: wiring objects together, listening on HTTP ports, talking to databases, doing security checks. **Spring Boot** is Spring + sensible defaults + an embedded Tomcat server — so a single `main()` method launches a full web app.

The 5 annotations you will see everywhere

Annotation	Meaning
`@SpringBootApplication`	Marks the `main()` class. Tells Spring "scan this package and start the app."
`@RestController`	"This class handles HTTP requests and returns JSON."
`@Service`	"This class contains business logic."
`@Repository`	"This class talks to the database." Usually extends `JpaRepository`.
`@Entity`	"This class maps to a database table."

A complete tiny controller

```
@RestController
@RequestMapping("/contracts")
public class ContractController {

    private final ContractService service;
    public ContractController(ContractService service) { this.service = service; }

    @GetMapping
    public List<Contract> list() {
        return service.findAll();
    }

    @PostMapping
    @PreAuthorize("hasAnyRole('ADMIN','RIGHTS_MANAGER')")
    public Contract create(@RequestBody @Valid Contract c) {
        return service.save(c);
    }
}
```

- `@RequestMapping("/contracts")` — base URL for this controller.
- `@GetMapping` / `@PostMapping` — map HTTP verb + path.
- `@RequestBody` — read JSON from the request and convert to a Java object.
- `@PreAuthorize` — only logged-in users with these roles may call this method.

4.2 Maven & pom.xml

Maven is the build tool. Every service has a `pom.xml` file declaring its dependencies. When you run `mvn spring-boot:run`, Maven downloads all the libraries from the internet and starts the app.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Common dependencies in this project:

- `spring-boot-starter-web` — REST endpoints
- `spring-boot-starter-data-jpa` — database access
- `spring-boot-starter-security` — auth/role checks
- `spring-cloud-starter-netflix-eureka-client` — register with Eureka
- `spring-cloud-starter-openfeign` — call other services
- `resilience4j-spring-boot3` — circuit breakers, retries
- `mysql-connector-j` — MySQL driver

4.3 application.properties — the config file

Each service has one. Typical entries:

```
spring.application.name=contract-service
server.port=8088

spring.datasource.url=jdbc:mysql://localhost:3306/contract_database
spring.datasource.username=root
spring.datasource.password=ctssql2026
spring.jpa.hibernate.ddl-auto=update

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

`ddl-auto=update` means Hibernate auto-creates and migrates tables on startup — fine for development, never for production.

4.4 The 10 services, one by one

① Service-Registry — port 8761

Plain Eureka server. Every other service phones home here on startup so the gateway can find them by name (`lb://contract-service`) instead of hardcoded IPs.

Open `http://localhost:8761` in your browser to see the live registry dashboard.

② API-Gateway — port 8082

Built with **Spring Cloud Gateway (WebFlux)**. Two jobs:

1. **Routing.** The gateway's config maps URL prefixes to services:
    - `/auth/**` , `/users/**` , `/roles/**` → Streamcast-Backend
    - `/api/titles/**` , `/api/assets/**` , `/api/metadata/**` → Content-Catalog
    - `/contracts/**` → Contract-Service
    - `/clauses/**` → Clause-Service
    - `/partners/**` → Partner-Service
    - `/manifests/**` → Distribution-Delivery
    - `/api/schedules/**` , `/api/conflicts/**` → schedule-service
    - `/api/usage/**` → usage-service
2. **JWT filter.** The `JwtAuthFilter` class reads the `Authorization` header, verifies the signature with the shared secret, extracts `email` + `role` , and forwards the request. If the token is missing/expired, the gateway sends 401 immediately. Login and register paths are public (no token needed).

③ Streamcast-Backend — port 8081

The auth + identity service. Owns two entities: `User` and `Role`.

Endpoints:

- `POST /auth/register` → creates a User and emails a verification link.
- `GET /auth/verify?token=...` → marks email as verified.
- `POST /auth/login` → returns a JWT.
- `POST /auth/forgot-password` / `/auth/reset-password` → password reset flow.
- `POST /auth/forgot-username` → emails the username.
- `/users`, `/users/{id}` — ADMIN-only CRUD over users.
- `/roles`, `/roles/{id}` — ADMIN-only CRUD over roles.

Roles supported: ADMIN, CONTENT_OWNER, RIGHTS_MANAGER, SCHEDULER, COMPLIANCE_OFFICER, DISTRIBUTION_OPERATOR, PARTNER_ADMIN. Every other service uses these role names in its `@PreAuthorize` rules.

④ Content-Catalog-Service — port 1020

The library of every piece of content. Three entities, all in `content_catalog_database`:

- **Title** — id, name, releaseDate, genre, language, status.
- **Asset** — a file belonging to a Title (VIDEO, AUDIO, SUBTITLE…). Holds `fileURI`, `checksum`, `duration`.
- **TitleMetadata** — flexible `key/value` pairs (director, cast, plot…).

Key endpoints:

- `POST /api/titles` — create a title (ADMIN, CONTENT_OWNER).
- `GET /api/titles` — list titles.
- `GET /api/titles/{id}/exists` — used by Distribution-Delivery to validate a title before shipping.
- `POST /api/assets/{titleId}` — attach an asset.
- `POST /api/titles/{titleId}/metadata` — add metadata.

Swagger UI is exposed at `/swagger-ui/index.html`.

⑤ Contract-Service — port 8088

One entity: **Contract**. Holds `titleId`, a JSON list of territories, start/end date, exclusivity flag, summary, and status.

Endpoints: standard CRUD on `/contracts`. All responses are wrapped in a generic envelope:

```
{
  "message": "Contract created successfully",
  "data": { ... },
  "success": true
}
```

Only ADMIN + RIGHTS_MANAGER can create or update; ADMIN can delete.

⑥ Clause-Service — port 8089

One entity: **Clause**. Each clause belongs to a contract (`contractId` FK) and carries a free-form `detailsJSON` string. Used to store things like "broadcast restriction in Region X from 2025-01-01 to 2026-01-01".

Endpoints: `POST /clauses/post`, `GET /clauses/get`.

⑦ Partner-Service — port 8087

One entity: **Partner**. Tracks the external broadcasters/platforms StreamCast delivers content to — their name, contact info, endpoint note, status, and the `titleId` they hold.

Validation rules: 2–50 char name, 10-digit phone, status ACTIVE/INACTIVE.

⑧ Distribution-Delivery — port 8083

The shipping department. Bundles a Title's Assets into a **Manifest** and tracks delivery to a Partner.

Entity: *Manifest* with titleId, assetIdsJSON (which assets are in the package), destination, createdBy, createdAt, status (PENDING/DELIVERED/FAILED), partnerId.

Endpoints:

- `POST /manifests` — create.
- `GET /manifests`, `GET /manifests/{id}`, `GET /manifests/partner/{partnerId}`.
- `POST /manifests/{id}/attempts` — log a delivery attempt.
- `POST /manifests/{id}/receipt` — partner confirms receipt.

This service uses Feign + Resilience4j. Before creating a manifest it calls Content-Catalog (`/api/titles/{id}/exists`) and Partner-Service (to read partner details). If either is down, a **circuit breaker** kicks in — after 50 % failures inside a 5-request window it "trips" and refuses to call for 10 seconds, returning a fallback. Retries: 3 attempts, 2 s apart.

⑨ Schedule-Service — port 1040

Two entities:

- **Schedule** — titleId, contractId, platform (NETFLIX, HULU, ZEE5...), startDateTime, endDateTime, windowtype (EXCLUSIVE / NON_EXCLUSIVE), status.
- **Conflict** — links two overlapping schedules; carries `resolved` flag + `detectedAt`.

Endpoints:

- `POST /api/schedules` — create.
- `GET /api/schedules/calendar` — paginated calendar with filters (status, platform).
- `POST /api/schedules/{id}/detect-conflicts` — manually trigger overlap detection.
- `GET /api/conflicts?scheduleId=...` — view conflicts.
- `PUT /api/conflicts/{id}/resolve` — mark resolved.

⑩ Usage-Service — port 1030

Read-only analytics. One entity: **UsageRecord** (titleId, platform, date, views, revenue).

Endpoints:

- `GET /api/usage/summary?start=...&end=...` — totals over a date range.
- `GET /api/usage/details?start=...&end=...` — every row in the range.
- `GET /api/usage/breakdown?start=...&end=...&groupBy=platform` — pivot by field.

4.5 How services call each other — OpenFeign

Distribution-Delivery does not hardcode `http://localhost:1020`. It declares a Feign interface and Spring builds a proxy:

```
@FeignClient(name = "content-catalog-service")
public interface TitleClient {
    @GetMapping("/api/titles/{id}/exists")
    Boolean exists(@PathVariable Integer id);
}
```

At call time Feign asks Eureka "where is content-catalog-service right now?" and gets back the live host:port. This decouples services and supports load-balancing across multiple instances.

4.6 Resilience4j — when a downstream is down

Wrap a Feign call in a circuit breaker:

```
@CircuitBreaker(name = "titleCircuitBreaker", fallbackMethod = "titleFallback")
@Retry(name = "titleRetry")
public boolean checkTitleExists(Integer id) {
    return titleClient.exists(id);
}

public boolean titleFallback(Integer id, Throwable t) {
    log.warn("title service unreachable, assuming title invalid");
    return false;
}
```

Three states: **CLOSED** (normal) → **OPEN** (too many failures, refuse for 10 s) → **HALF_OPEN** (try a few, decide where to go next).

5. Frontend Deep Dive

5.1 What is Angular?

Angular is a **JavaScript framework for building single-page web applications (SPAs)**. "Single page" means the browser only ever loads `index.html` once; after that, Angular swaps content in and out without a full page reload.

Angular is written in **TypeScript** — JavaScript with types added on top — and is organised around four building blocks:

1. **Components** — a chunk of UI (HTML + CSS + a TypeScript class). Like a Lego brick.
2. **Services** — plain classes that hold logic / talk to APIs. Shared by many components.
3. **Routing** — maps URLs (`/titles`) to components.
4. **Guards / Interceptors** — middleware that runs before navigation or HTTP calls.

5.2 TypeScript essentials in 60 seconds

```
// type annotation on a variable
let count: number = 0;

// interface — a shape contract
interface Title {
  id: number;
  name: string;
  releaseDate: string;
  genre: string;
}

// class with constructor injection
class TitlesService {
  constructor(private http: HttpClient) {}
  list(): Observable<Title[]> {
    return this.http.get<Title[]>('/api/titles');
  }
}
```

Key ideas: explicit types, interfaces describe data shapes, `private http` in the constructor automatically saves the parameter as a field, and the compiler catches mistakes before the code runs.

5.3 A component, dissected

```
@Component({
  selector: 'app-titles',
  standalone: true,
  imports: [CommonModule, FormsModule, RouterLink],
  template: `
    <h2>Titles</h2>
    <button (click)="reload()">Refresh</button>
    <ul>
      <li *ngFor="let t of titles()">
        <a [routerLink]="['/titles', t.id]">{{ t.name }}</a>
      </li>
    </ul>
  `,
})
export class TitlesComponent {
  titles = signal<Title[]>([]);
  constructor(private svc: TitlesService) { this.reload(); }
  reload() {
    this.svc.list().subscribe(rows => this.titles.set(rows));
  }
}
```

Things to notice:

- `standalone: true` — Angular 17 modern style; no NgModule needed.
- `selector: 'app-titles'` — this becomes the HTML tag `<app-titles></app-titles>`.
- `*ngFor="let t of titles()"` — loop a template; calling `titles()` reads the Signal.
- `(click)="reload()"` — event binding (parentheses = event, square brackets = property).
- `{{ t.name }}` — interpolation (display a value).
- The constructor injects the service. Angular looks at the parameter type and supplies the right instance.

5.4 Routing

In `app.routes.ts`:

```
export const routes: Routes = [
  { path: 'login', component: LoginComponent },
  { path: '', component: ShellComponent, canActivate: [authGuard], children: [
      { path: 'dashboard', component: DashboardComponent },
      { path: 'titles', component: TitlesComponent,
        canActivate: [roleGuard(['ADMIN','CONTENT_OWNER','RIGHTS_MANAGER','SCHEDULER'])] },
      { path: 'titles/:id', component: TitleDetailComponent },
      { path: 'contracts', component: ContractsComponent,
        canActivate: [roleGuard(['ADMIN','RIGHTS_MANAGER'])] },
      // ... clauses, partners, manifests, schedules, conflicts, usage,
      //     users (ADMIN only), roles (ADMIN only)
  ]},
  { path: 'forbidden', component: ForbiddenComponent },
  { path: '**', redirectTo: 'dashboard' }
];
```

- **authGuard** blocks every authenticated route unless a JWT is present.
- **roleGuard(['ADMIN', ...])** reads the role claim and bounces unauthorized users to `/forbidden`.

5.5 HttpClient + Auth Interceptor

Every service uses Angular's `HttpClient`:

```
@Injectable({ providedIn: 'root' })
export class ContractsService {
  private base = `${environment.gateway}/contracts`;
  constructor(private http: HttpClient) {}

  list(): Observable<Contract[]> {
    return this.http.get<ApiResponse<Contract[]>>(this.base)
                    .pipe(map(r => r.data));
  }
}
```

The Auth Interceptor automatically attaches the JWT:

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('jwt');
  const cloned = token ? req.clone({
    setHeaders: { Authorization: `Bearer ${token}` }
  }) : req;
  return next(cloned);
};
```

Registered in `app.config.ts` via `provideHttpClient(withInterceptors([authInterceptor]))`.

5.6 Signals — Angular 17's new reactivity

A Signal is a value you can read and watch. When you change it, every template using it re-renders automatically.

```
const count = signal(0);
count();          // read → 0
count.set(5);     // write → 5
count.update(n => n + 1);

const doubled = computed(() => count() * 2);
effect(() => console.log('count is now', count()));
```

StreamCast uses Signals to store the current user, the titles list, schedule filters etc. They replace older patterns like `BehaviorSubject` and manual change detection.

5.7 Tailwind CSS & Angular Material

Two complementary styling layers are used:

- **Tailwind CSS** — utility classes you compose inline:

  ```
  <div class="flex items-center gap-4 p-4 bg-white rounded-lg shadow">
    <h2 class="text-xl font-semibold text-slate-800">Titles</h2>
  </div>
  ```

  Each class is one CSS property. No more naming things like `.title-card-header`.
- **Angular Material** — pre-built UI components (`mat-button`, `mat-dialog`, `mat-table`, `mat-paginator`...) you import into a component and use directly. They handle keyboard / accessibility for you.

5.8 Folder-by-folder tour of `streamcast-ui`

Path	What's there
`angular.json`	Angular workspace config (build options, assets, styles).
`tailwind.config.js`	Where Tailwind scans for class names & theme tokens.
`package.json`	npm dependencies and run scripts (`npm start` = `ng serve`).
`src/main.ts`	Entry point: bootstraps `AppComponent` with `appConfig` .
`src/app/app.routes.ts`	All route definitions + guards.
`src/app/app.config.ts`	Global providers (router, HttpClient with interceptor, Material animations).
`src/app/core/auth/`	`AuthService` , `authGuard` , `roleGuard` , JWT decode utility.
`src/app/core/http/auth.interceptor.ts`	Attaches Bearer token to every outgoing HTTP request.
`src/app/core/models/`	TypeScript interfaces — `Title` , `Contract` , `Schedule` , `ApiResponse<T>` , etc.
`src/app/features/auth/login.component.ts`	Login form, calls `POST /auth/login` , stores JWT.
`src/app/features/dashboard/`	Landing page after login.
`src/app/features/catalog/`	Titles + Assets management UI.
`src/app/features/contracts/`	Contract CRUD pages + service.
`src/app/features/clauses/`	Clause CRUD.
`src/app/features/partners/`	Partner CRUD.
`src/app/features/manifests/`	Manifest creation + delivery tracking.
`src/app/features/schedules/`	Calendar view, create / update / delete schedules.
`src/app/features/conflicts/`	List + resolve schedule conflicts.
`src/app/features/usage/`	Reports / charts of views & revenue.
`src/app/features/admin/`	Users + Roles screens (ADMIN-only).
`src/app/features/misc/forbidden.component.ts`	403 page.
`src/app/layout/shell.component.ts`	The frame around every authenticated page — top bar, side nav, router outlet.

6. The User Journey — End to End

Let's follow one click: **"User logs in, opens Titles page, clicks a title, schedules it."**

① Login

1. Browser loads `http://localhost:4200/login`.
2. User types email + password and submits the form.
3. `LoginComponent` calls `AuthService.login()`.
4. That hits `POST http://localhost:8082/auth/login` (the gateway).
5. Gateway sees this path is in the "public" list, forwards to Streamcast-Backend on 8081.
6. Streamcast-Backend checks the password (BCrypt), builds a JWT containing `sub=email` + `role=ADMIN`, signs it with the secret, and returns it.
7. Angular stores it in `localStorage` and routes the user to `/dashboard`.

② Open the Titles page

1. User clicks "Titles" in the side nav → router navigates to `/titles`.
2. `authGuard` checks "is a JWT present?" — yes.
3. `roleGuard(['ADMIN','CONTENT_OWNER','RIGHTS_MANAGER','SCHEDULER'])` reads the role claim — passes.
4. `TitlesComponent` mounts. In its constructor it calls `TitlesService.list()`.
5. `HttpClient` sends `GET /api/titles`; the auth interceptor adds `Authorization: Bearer ...`.
6. Gateway validates JWT, then forwards to Content-Catalog (1020).
7. Content-Catalog's `TitleController.list()` runs, JPA repository fetches rows, JSON returned.
8. The Signal is updated, Angular re-renders the list.

③ Open a title's details

1. User clicks a row → router goes to `/titles/42`.
2. `TitleDetailComponent` fires three parallel calls:
    - `GET /api/titles/42`
    - `GET /api/titles/42/assets`
    - `GET /api/titles/42/metadata`
3. All three resolve, the page shows title info + asset list + metadata table.

④ Schedule this title

1. User clicks "Add Schedule" → opens a Material dialog.
2. Fills startDateTime / endDateTime / platform / windowtype / contractId.

3. `POST /api/schedules` is sent.
4. schedule-service validates the body, inserts a row in `schedules`, runs conflict detection (overlapping time ranges on same title or platform). If any are found, rows are written to `conflicts`.
5. Response goes back up; UI navigates to the calendar showing the new entry.
6. If conflicts exist, the `/conflicts` page lights up — user can resolve them via `PUT /api/conflicts/{id}/resolve`.

7. How to Run It Locally

Prerequisites

- JDK 21 (covers both Java 17 and 21 services).
- Maven 3.9+.
- MySQL 8.x running on `localhost:3306` with user `root` / password `ctssql2026`.
- Node.js 18+ and npm.
- An IDE — IntelliJ IDEA recommended.

Step-by-step

1. Start MySQL. Create the databases (Hibernate `ddl-auto=update` will build tables but it does not create the databases themselves):

```
CREATE DATABASE streamcast_database;
CREATE DATABASE content_catalog_database;
CREATE DATABASE contract_database;
CREATE DATABASE clause_database;
CREATE DATABASE partner_database;
CREATE DATABASE distribution_delivery_database;
CREATE DATABASE schedule_database;
CREATE DATABASE usage_database;
```

2. Start Service-Registry first:

```
cd Service-Registry
mvn spring-boot:run
```

   Open `http://localhost:8761` — you should see the Eureka dashboard.

3. Start each service (each in its own terminal):

```
cd Streamcast-Backend       && mvn spring-boot:run    # 8081
cd Content-Catalog-Service  && mvn spring-boot:run    # 1020
cd Contract-Service         && mvn spring-boot:run    # 8088
cd Clause-Service           && mvn spring-boot:run    # 8089
cd Partner-Service          && mvn spring-boot:run    # 8087
cd Distribution-Delivery    && mvn spring-boot:run    # 8083
cd schedule-service         && mvn spring-boot:run    # 1040
cd usage-service            && mvn spring-boot:run    # 1030
```

4. Start the API-Gateway last (it can boot before the services, but routes will fail until they register):

```
cd API-Gateway && mvn spring-boot:run     # 8082
```

5. Start the frontend:

```
cd streamcast-ui
npm install
npm start                                  # serves on http://localhost:4200
```

6. Register an ADMIN user via `POST http://localhost:8082/auth/register`, verify the email link from the SMTP logs, then log in via the UI.

8. Common Troubleshooting

Symptom	Likely cause	Fix
UI returns 401 on every call	JWT missing or expired	Log in again — token survives in `localStorage`, check DevTools → Application.
UI returns 403	Your user's role can't reach that endpoint	Check `@PreAuthorize` in the controller; promote the role via `PUT /users/{id}`.
Gateway returns 503	Downstream service not registered in Eureka yet	Wait 30 s after starting; check `http://localhost:8761`.
"Connection refused" in service logs	MySQL not running, or wrong password	Start MySQL; verify `application.properties` credentials.
CORS error in browser console	Calling a service directly instead of via gateway	Always call `http://localhost:8082/...`; gateway adds CORS headers.
Hibernate "table doesn't exist"	Database wasn't created	Run the CREATE DATABASE statements above.
Distribution-Delivery hangs / returns fallback	Circuit breaker tripped for Content-Catalog or Partner	Restart the downstream and wait 10 s for the breaker to half-open.
Email never arrives during registration	SMTP creds in Streamcast-Backend	Check `spring.mail.*` properties — use an app-specific password for Gmail.

9. What to Learn Next — A Roadmap

If you are completely new to backend:

1. Java basics — classes, methods, generics, lambdas.
2. Spring Boot — Baeldung "Spring Boot Tutorial", official guide "Building a RESTful Web Service".
3. JPA / Hibernate — entities, relationships, JPQL.
4. Spring Security — UserDetails, filters, role hierarchy, JWT.
5. Spring Cloud — Eureka, Gateway, OpenFeign, Resilience4j.

If you are completely new to frontend:

1. HTML/CSS/JS fundamentals — MDN tutorials.
2. TypeScript Handbook (typescriptlang.org) — at least chapters 1–4.
3. Angular official tour: *angular.dev/tutorials/learn-angular*.
4. RxJS basics — Observables, `map`, `switchMap`, `tap`.
5. Angular 17 Signals — *angular.dev/guide/signals*.
6. Tailwind CSS — *tailwindcss.com/docs/utility-first*.

Bring it together — practical exercises on this codebase

- Add a new field to `Contract` (e.g. `currency`) — update entity, repository (free), DTO, controller, frontend service + form.
- Add a new role `VIEWER` that can only do GET on titles. Update Streamcast-Backend role table + Spring Security configs in each service + the frontend role guard.
- Add a new microservice `notification-service` that exposes `POST /notifications` and is called by Distribution-Delivery whenever a delivery succeeds.
- Replace one of the Tailwind dashboards with Angular Material's `mat-table` + `mat-paginator`.

> **You're done!** Read this once front-to-back, then keep it open as a reference while you poke around the code. The fastest way to learn is to pick one service, run it, hit it with Postman, and step through the controller in the debugger. After 2-3 services it all clicks.